Friday Idea, Sunday Cheers, Monday Launch.

Dilum De Silva
Google Developer Expert
Senior Engineer at Resolution Farming

Stitch BETA





The Old Weekend Friction

⚠️ Manual boilerplate

Folder structure, architecture, state management, routing

⚠️ Pixel-pushing custom layouts

Theme, layouts, then container grids that overflows

⚠️ Context drift & setup hell

Broken dependencies, endless debugging instead validating the product

The Next-Gen Velocity

🛠️ Google Stitch (Beta)

Raw intent to polished UI

🛠️ PRD + MCP + Skills

PRD for living spec, MCP for tooling and skills for best practices

🛠️ Google Antigravity

Autonomous agent first IDE and tools

The Blueprint (PRD as the Source of Truth)

Why it matters:

Autonomous agents excel when given absolute technical boundaries.

The Strategy:

Spend 5-15 minutes chatting with any AI (Gemini) to output a highly concise **PRD.md** straight into your project **/docs** folder.

The Blueprint: Product Requirement Document (PRD)

Define core features, color palettes, data schemas, and folder structures *before* writing code.

Traditional PRD

Written for People: Designed to align Product, Design, and Engineering teams on *what* to build and *why*.

High-Level Scope: Focuses heavily on user personas, abstract user flows, functional requirements, and business rules.

Decoupled Architecture: Intentionally leaves the technical design patterns, directory structures, and visual layouts to separate documentation or Figma specs.

Table 1: Generic PRD Template Outline	
1.0	Scope of the document
2.0	Applicable and Reference Documents
3.0	Requirements
3.1	Product Scope
3.1.1	Business Case, Mission or Vision, Goals, Objectives
3.1.2	Product Description
3.1.3	Product Architecture and Core Components
3.1.4	Operational Concepts, Scenarios, Workflows, Use Cases
3.1.5	Context Diagram, Interfaces
3.2	Design Input Requirements
4.0	System Verification (Criteria for Acceptance, Method)
Appendix A Acronymns	
Appendix B Glossary	
Appendix C Traceability Matrix	
Appendix D TBDs/TBRs	

Agentic (AI) PRD

Written for Execution Engines: A high-density, unambiguous technical spec designed for autonomous agents like Antigravity to parse and execute directly.

Architectural Guardrails: Explicitly enforces your technical stack up front.

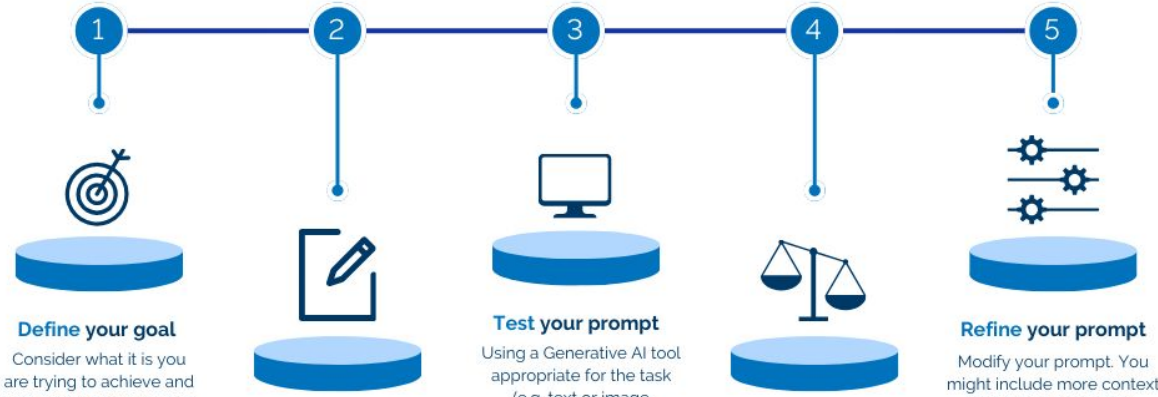
The Unified Bridge (MCP): Connects product logic to visual designs in Google Stitch via MCP, allowing the agent to bind code to real design tokens.

Deterministic Data Mocking: Hardcodes localized data models and seed schemas into the document to ensure the agent builds resilient, real-world layouts instead of empty layout containers.

“Prompting isn't creative writing. It’s a modern-day engineering process.”

Newcastle University

Prompting as a Process



Define your goal

Consider what it is you are trying to achieve and what output you want to be generated.

Draft an initial prompt

Prompting is an iterative process so you might need to experiment with different styles of prompt, but starting with an initial prompt gives you something to work from.

Test your prompt

Using a Generative AI tool appropriate for the task (e.g. text or image generation) test your prompt.

Evaluate the output

Now is the time to apply your critical skills. Consider the accuracy, relevance and reliability of the output. Has it achieved your defined goal. If not, now is the time to move to the next stage in the process.

Refine your prompt

Modify your prompt. You might include more context or detailed instruction, outline a different format to generate or change the focus of your task slightly.

Prompting is an iterative process where you will need to repeat the evaluate and refine steps until you get to the point where you are happy with the output.

Expand your prompt into multiple layers

[Give a role]
[Define the goal]
[Provide context]
[Does it need AI?]
[Create the vibe]
Optional : [Add a visual]



Good prompt

INSTEAD OF THIS

Help me build an app to track my food and water intake.

Powerful prompt

TRY THIS

You're an app builder who prioritizes good user experience. Create a web app that helps me track my food calories and water intake. I plan to use this app daily to count my calories and track my water intake and I want weekly summary. It would be good to use AI to track my calories if I can provide pictures of my food. Create a skeuomorphic vibe. Use the attached mockup for inspiration.

What are Agent Skills?

The MCPs:

MCP gives the agent the **eyes and hands** to talk to external tools (like checking a directory or connecting to Stitch).

The Skills:

A Skill teaches the agent the **industry best-practice workflow** on *how* to execute a specific task.

Eg: github.com/flutter/skills

Where do I keep my Skills?

```
garage-sale/
├── .agents/                    <-- Added by the script
│   └── skills/                 <-- Subdirectory
│       ├── flutter-apply-architecture-best-practices/
│       ├── flutter-fix-layout-issues/
│       └── [other selected skills...]
├── docs/
│   └── prd.md                  <-- Your Agentic PRD
└── lib/                        <-- Your actual Flutter code
```

Stitch & Antigravity Prompts



Learnings from the workflow

- Human-in-the-Loop
  - Agent ready PR is a huge win to break the loop
- Standardized agent skills over prompts
- Near real world team orchestration with PRD, MCP & Skills
- Real (mocked) data catches many layout-breaks, bugs

Limitations

- Review generated code, it's a tool, not a replacement for understanding.
- Complex business logic still needs human judgment.
- Tools require saving often, verifying always and sometimes restoring context.

"Use these where speed matters and correctness is easy to check. Be hands-on where the logic is subtle.

“The gap between idea and app has never been smaller. Design it in Stitch. Build it in Antigravity. Ship it in Flutter. Go try it tonight.”

Thank you!

Talk materials available at dilumdesilva.dev

